

1. Dependencias actuales

La tercera columna se limita a justificar la existencia de la dependencia: herencia, realización de interfaz, referencia almacenada, tipo usado en una firma, creación de un objeto o consulta explícita de otro tipo.

Alto nivel: reglas, contratos y dominio

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
ReglaPotrero	Sin dependencias propias relevantes.	No declara herencia, campos, parámetros ni usos de otros tipos propios del sistema.	Bib_Hacienda/Reglas/ReglaPotrero.cs, líneas 9–12.
ReglaRes	Sin dependencias propias relevantes.	No declara herencia, campos, parámetros ni usos de otros tipos propios del sistema.	Bib_Hacienda/Reglas/ReglaRes.cs, líneas 9–21.
ReglaVacuna	Sin dependencias propias relevantes.	No declara herencia, campos, parámetros ni usos de otros tipos propios del sistema.	Bib_Hacienda/Reglas/ReglaVacuna.cs, líneas 9–22.
IAutenticacion	Usuario.	Usuario forma parte de la firma del contrato de autenticación/autorización.	Bib_Hacienda/Interfaces/IAutenticacion.cs, líneas 11–14.
ICreacionVacuna	Viva.enum_I_atenuaciones.	El enum de atenuación aparece como tipo de parámetro en las operaciones de creación de vacuna.	Bib_Hacienda/Interfaces/ICreacionVacuna.cs, líneas 10–15.
IVacunacion	Vacuna.	Vacuna aparece como tipo de parámetro en la operación de vacunación.	Bib_Hacienda/Interfaces/IVacunacion.cs, líneas 10–13.
IValidarInformacion	Res; Potrero; Vacuna; Venta.	Cada uno de esos tipos aparece como parámetro de una operación de validación del contrato.	Bib_Hacienda/Interfaces/IValidarInformacion.cs, líneas 11–23.
IVentaRes	Sin dependencia propia a otra clase del sistema.	La firma utiliza identificadores, monto y tipos básicos; no declara tipos propios como parámetros o retornos.	Bib_Hacienda/Interfaces/IVentaRes.cs, líneas 10–13.
Vacuna	Sin dependencias propias de otras clases del sistema.	No hereda de tipos propios ni mantiene referencias a otras clases del sistema.	Bib_Hacienda/Clases/Vacuna.cs, líneas 10–32.

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
Bacteriana	Vacuna; ReglaVacuna.	Depende de Vacuna por herencia y de ReglaVacuna porque consulta sus límites al validar el período de aplicación.	Bib_Hacienda/Clases/Bacteriana.cs, líneas 10, 17 y 23–25.
Viva	Vacuna.	Depende de Vacuna por herencia. El enum de atenuaciones es un tipo anidado de la propia clase.	Bib_Hacienda/Clases/Viva.cs, líneas 9–24.
Res	Vacuna.	Mantiene una colección tipada como Vacuna y expone esa colección como parte de su estado.	Bib_Hacienda/Clases/Res.cs, líneas 10, 17, 22–27 y 36.
Ternero	Res; ReglaRes.	Depende de Res por herencia y de ReglaRes porque consulta el límite de edad correspondiente.	Bib_Hacienda/Clases/Ternero.cs, líneas 10, 14 y 19–23.
Cebon	Res; ReglaRes.	Depende de Res por herencia y de ReglaRes porque consulta los límites que determinan su rango de edad.	Bib_Hacienda/Clases/Cebon.cs, líneas 10, 14 y 19–23.
Novillo	Res; ReglaRes.	Depende de Res por herencia y de ReglaRes porque consulta el límite de edad utilizado en su validación.	Bib_Hacienda/Clases/Novillo.cs, líneas 10, 14 y 19–23.
Potrero	Res; Ternero; Cebon; Novillo; ReglaPotrero; ReglaRes; PublisherPotreroMitad; PublisherPotreroLleno; PublisherPesoVenta; PublisherPesoMin.	Mantiene una lista de Res; crea Ternero/Cebon/Novillo; consulta ReglaPotrero y ReglaRes; y crea/retiene publishers que usa en sus operaciones.	Bib_Hacienda/Clases/Potrero.cs, líneas 15–24, 30–33, 53–100, 109–138 y 164–203.
Hacienda	Potrero; Venta; Vacuna; Res; Bacteriana; Viva; Ternero; Cebon; Novillo; ReglaVacuna; I_tipos_potreros; enum_I_atenuaciones; Publishers; IVacunacion; IVentaRes; ICreacionVacuna.	Mantiene colecciones de Potrero/Venta/Vacuna; crea y consulta tipos concretos del dominio; usa ReglaVacuna y enums; crea/retiene publishers; y realiza tres interfaces propias.	Bib_Hacienda/Clases/Hacienda.cs, líneas 16–21, 43–57, 61–79, 127–160, 220–250, 267–322, 335–429 y 451–547.

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
Venta	Potrero; Res.	Conserva referencias de tipo Potrero y Res como parte de sus datos.	Bib_Hacienda/Clases/Venta.cs, líneas 9–27.
Usuario	Sin dependencias propias relevantes.	No hereda de tipos propios ni mantiene referencias a otras clases del sistema.	Bib_Hacienda/Clases/Usuario.cs, líneas 9–21.
Autenticacion	IAutenticacion; Usuario.	Realiza IAutenticacion y mantiene/recibe objetos Usuario en su colección y operaciones.	Bib_Hacienda/Clases/Autenticacion.cs, líneas 10–25, 44–52, 63–73, 76–94 y 102–146.
Validacion	IValidarInformacion; Res; Potrero; Vacuna; Venta.	Realiza IValidarInformacion y declara operaciones cuyos parámetros son Res, Potrero, Vacuna y Venta.	Bib_Hacienda/Clases/Validaciones/Validacion.cs, líneas 10–17.
ValidadorPotrero	Validacion; Potrero; Res; Vacuna; Venta.	Hereda de Validacion y sobrescribe las cuatro firmas; Potrero es el tipo que procesa en su validación concreta.	Bib_Hacienda/Clases/Validaciones/ValidarPotrero.cs, líneas 9–33.
ValidadorRes	Validacion; Res; Potrero; Vacuna; Venta.	Hereda de Validacion y sobrescribe las cuatro firmas; Res es el tipo que procesa en su validación concreta.	Bib_Hacienda/Clases/Validaciones/ValidarRes.cs, líneas 9–33.
ValidadorVacuna	Validacion; Vacuna; Res; Potrero; Venta.	Hereda de Validacion y sobrescribe las cuatro firmas; Vacuna es el tipo que procesa en su validación concreta.	Bib_Hacienda/Clases/Validaciones/ValidarVacuna.cs, líneas 9–33.
ValidadorVenta	Validacion; Venta; Res; Potrero; Vacuna.	Hereda de Validacion y sobrescribe las cuatro firmas; Venta es el tipo que procesa en su validación concreta.	Bib_Hacienda/Clases/Validaciones/ValidarVenta.cs, líneas 9–33.
PublisherPesoMin	Res; Ternero; Cebon; Novillo; ReglaRes; PublisherPesoVenta.	Recibe/evalúa Res, distingue sus tipos concretos, consulta ReglaRes y declara una conversión desde PublisherPesoVenta.	Bib_Hacienda/Eventos/PublisherPesoMin.cs, líneas 11–18, 23–30 y 50–52.
PublisherPesoVenta	Res; Ternero; Cebon; Novillo; ReglaRes.	Recibe/evalúa Res, distingue sus tipos concretos y consulta los valores definidos en ReglaRes.	Bib_Hacienda/Eventos/PublisherPesoVenta.cs, líneas 11–18 y 24–34.

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
PublisherPotreroLleno	Potrero; ReglaPotrero.	Recibe un Potrero y compara su cantidad de reses con el límite definido en ReglaPotrero.	Bib_Hacienda/Eventos/PublisherPotreroLleno.cs, líneas 11–25.
PublisherPotreroMitad	Potrero; ReglaPotrero.	Recibe un Potrero y usa ReglaPotrero para calcular el punto de mitad de capacidad.	Bib_Hacienda/Eventos/PublisherPotreroMitad.cs, líneas 11–28.
PublisherVacunaVencida	Vacuna.	Recibe y consulta datos de Vacuna para determinar el evento de vencimiento.	Bib_Hacienda/Eventos/PublisherVacunaVencida.cs, líneas 10–29 y 31–51.
PublisherVacunacionCompletada	Res; Ternero; Cebon; Novillo; ReglaVacuna.	Recibe una Res, distingue su tipo concreto y consulta ReglaVacuna para evaluar el esquema aplicado.	Bib_Hacienda/Eventos/PublisherVacunacionCompletada.cs, líneas 11–18 y 27–40.

Nivel de aplicación

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
PotreroService	Hacienda; PersistenciaService; Potrero; I_tipos_potreros.	Hacienda y PersistenciaService se reciben y almacenan en campos; Potrero y I_tipos_potreros se usan en las operaciones de gestión.	p_mvchacienda/Servicios/PotreroService.cs, líneas 6–16, 20–35, 51–62, 70–95 y 110–120.
ResService	Hacienda; PersistenciaService; Potrero; Res; Ternero; Cebon; Novillo.	Hacienda y PersistenciaService se reciben y almacenan; las operaciones recorren Potrero/Res y distinguen Ternero, Cebon y Novillo.	p_mvchacienda/Servicios/ResService.cs, líneas 5–15, 18–34, 37–45 y 53–69.
VacunaService	Hacienda; PersistenciaService; Vacuna; Bacteriana; Viva; enum_I_atenuaciones.	Hacienda y PersistenciaService se reciben y almacenan; las operaciones usan Vacuna y distinguen Bacteriana/Viva, además del enum de atenuación.	p_mvchacienda/Servicios/VacunaService.cs, líneas 6–15, 18–44, 53–92, 100–125 y 128–145.

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
VentaService	Hacienda; PersistenciaService; Venta; Potrero.	Hacienda y PersistenciaService se reciben y almacenan; las operaciones consultan Venta y acceden al Potrero asociado.	p_mvcHacienda/Servicios/VentaService.cs, líneas 5–15, 17–31 y 34–57.
UsuarioService	PersistenciaService; Usuario; Claim; Task.	PersistenciaService se recibe y almacena; Usuario se mantiene en la colección; Claim se crea en autenticación y Task aparece en el retorno asíncrono.	p_mvcHacienda/Servicios/UsuarioService.cs, líneas 7–21, 24–49, 60–79 y 92–106.

Bajo nivel y composición técnica

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
InterceptorAutenticacion	IInterceptor; IHttpContextAccessor; IInvocation; Usuario.	Realiza IInterceptor; recibe y almacena IHttpContextAccessor; IInvocation llega a Intercept; Usuario se inspecciona entre los argumentos interceptados.	Bib_Hacienda/Aspectos/InterceptorAutenticacion.cs, líneas 12–23, 28–35 y 40–65.
InterceptorValidarInformacion	IInterceptor; IHttpContextAccessor; IInvocation.	Realiza IInterceptor; recibe y almacena IHttpContextAccessor; IInvocation se usa como parámetro de la operación interceptada.	Bib_Hacienda/Aspectos/InterceptorValidarInformacion.cs, líneas 11–22, 24–41 y 43–80.
PersistenciaService	IHttpContextAccessor; IWebHostEnvironment; InterceptorValidarInformacion; ProxyGenerator; ValidadorVacuna; ValidadorPotrero; ValidadorRes; ValidadorVenta; Potrero; Res; Ternero; Cebon; Novillo; Venta; Vacuna; Bacteriana; Viva; Usuario; I_tipos_potreros; enum_I_atenuaciones.	Recibe abstracciones de entorno/contexto; crea y retiene interceptor/proxies; y usa tipos del dominio como parámetros, colecciones y objetos que serializa o reconstruye.	p_mvcHacienda/Servicios/PersistenciaService.cs, líneas 12–55, 61–83, 94–121, 132–206, 217–289, 303–378, 389–442, 454–517, 529–589 y 601–627.
LoginViewModel	DataAnnotations de .NET.	Sus propiedades están anotadas con atributos de validación/presentación de .NET; no referencia clases propias del dominio.	p_mvcHacienda/Models/LoginViewModel.cs, líneas 5–12.

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
ErrorViewModel	Tipos base de .NET.	Solo usa string y una propiedad calculada; no referencia clases propias del dominio.	p_mvcHacienda/Models/ErrorViewModel.cs, líneas 3–7.
AccountController	Controller; UsuarioService; LoginViewModel; IActionResult; Task<IActionResult>; ClaimsPrincipal; ClaimsIdentity.	Hereda de Controller; almacena UsuarioService; recibe LoginViewModel; retorna tipos MVC/asíncronos y crea ClaimsPrincipal/ClaimsIdentity.	p_mvcHacienda/Controllers/AccountController.cs, líneas 8–18, 24–44 y 47–55.
HomeController	Controller; ILogger<HomeController>; IActionResult; ErrorViewModel; Activity.	Hereda de Controller; almacena ILogger; retorna IActionResult y usa ErrorViewModel/Activity en la acción de error.	p_mvcHacienda/Controllers/HomeController.cs, líneas 8–16 y 19–32.
PotreroController	Controller; PotreroService; Hacienda; PersistenciaService; Potrero; I_tipos_potreros; ActionResult.	Hereda de Controller; almacena PotreroService, Hacienda y PersistenciaService; las acciones usan Potrero, el enum y retornos MVC.	p_mvcHacienda/Controllers/PotreroController.cs, líneas 8–20, 27–34, 45–56 y 63–80.
ResController	Controller; ResService; PotreroService; Hacienda; PersistenciaService; Potrero; Res; Vacuna; ActionResult; CultureInfo; NumberStyles.	Hereda de Controller; almacena dos Services, Hacienda y PersistenciaService; las acciones usan entidades de dominio y tipos de parseo/retorno MVC.	p_mvcHacienda/Controllers/ResController.cs, líneas 8–23, 27–54, 65–89, 105–116 y 131–169.
UsuarioController	Controller; UsuarioService; Usuario; ActionResult.	Hereda de Controller; almacena UsuarioService; las acciones reciben/entregan Usuario y retornan ActionResult.	p_mvcHacienda/Controllers/UsuarioController.cs, líneas 6–24 y 27–48.
VacunaController	Controller; VacunaService; ResService; PotreroService; Vacuna; Res; Potrero; enum_I_atenuaciones; ActionResult; CultureInfo; DateTimeStyles.	Hereda de Controller; almacena tres Services; las acciones usan entidades/enums del dominio y tipos MVC/globalización.	p_mvcHacienda/Controllers/VacunaController.cs, líneas 9–22, 24–50, 53–79, 94–115 y 139–169.
VentaController	Controller; VentaService; Venta; ActionResult; IFormCollection.	Hereda de Controller; almacena VentaService; las acciones usan Venta, retornos MVC e IFormCollection.	p_mvcHacienda/Controllers/VentaController.cs, líneas 7–24, 27–54 y 66–80.

Elemento	Depende actualmente de	Por qué existe la dependencia	Evidencia en el código
Program	WebApplication; WebApplicationBuilder; IServiceCollection; IServiceProvider; IHttpContextAccessor; PersistenciaService; Hacienda; PotreroService; ResService; VacunaService; VentaService; UsuarioService.	Usa tipos de ASP.NET para construir/configurar la aplicación y referencia las clases concretas que registra, crea o resuelve en el contenedor.	p_mvcHacienda/Program.cs, líneas 7–14, 16–36, 38–74, 76–89 y 91–112.

2. Clasificación por nivel

Alto nivel. Incluye ReglaPotrero, ReglaRes y ReglaVacuna; las entidades y especializaciones del dominio (Hacienda, Potrero, Res, Vacuna, Venta, Usuario, Ternero, Cebon, Novillo, Bacteriana y Viva); los contratos propios IAutenticacion, ICreacionVacuna, IVacunacion, IValidarInformacion e IVentaRes; las clases de validación; y los publishers de eventos. Se agrupan aquí porque sus relaciones están expresadas en conceptos propios de la hacienda: reses, potreros, vacunas, ventas, usuarios, pesos, edades, capacidad, vacunación y validación.

Nivel de aplicación. PotreroService, ResService, VacunaService, VentaService y UsuarioService coordinan operaciones entre el dominio y otros componentes del sistema. Sus dependencias muestran que se sitúan entre los objetos de negocio y la persistencia/presentación.

Bajo nivel. PersistenciaService, los interceptores, los Controllers y los Models dependen de mecanismos específicos de archivos, ASP.NET MVC, HTTP, Castle DynamicProxy, logging, claims, formularios y tipos de presentación. Son los elementos más ligados a la tecnología concreta usada por la aplicación.

Program. Se mantiene separado de la clasificación anterior como punto de composición: referencia los componentes que registra, construye y conecta durante el arranque de la aplicación.

Conclusiones de la clasificación actual:

- Las reglas ReglaPotrero, ReglaRes y ReglaVacuna constituyen la política más independiente del sistema: otras clases las consultan, pero ellas no dependen de clases propias.

- El dominio se relaciona internamente mediante herencia, asociaciones, composición de publishers y uso de reglas; no referencia Controllers ni PersistenciaService.
- Los Services dependen simultáneamente de objetos del dominio y de PersistenciaService, por lo que conectan el nivel de aplicación con ambos lados del sistema.
- PersistenciaService depende de tipos del dominio para guardar y reconstruir información, además de tipos técnicos de ASP.NET y Castle.
- Los Controllers dependen de Services y tipos MVC; PotreroController y ResController también mantienen referencias directas a Hacienda y PersistenciaService.

3. Dirección de dependencias e inversión observable hoy

Lectura de las fronteras actuales

Entre aplicación e infraestructura

Los cinco Services de aplicación mantienen una referencia directa a PersistenciaService. Por tanto, en esta frontera la dependencia está expresada desde aplicación hacia una clase concreta de infraestructura.

- PotreroService → PersistenciaService.
- ResService → PersistenciaService.
- VacunaService → PersistenciaService.
- VentaService → PersistenciaService.
- UsuarioService → PersistenciaService.

En esta frontera no se observa una abstracción propia entre los Services y PersistenciaService. La dependencia actual se declara directamente contra PersistenciaService.

Evidencia: p_mvcHacienda/Servicios/PotreroService.cs, líneas 6–16; ResService.cs, líneas 5–15; VacunaService.cs, líneas 6–15; VentaService.cs, líneas 5–15; UsuarioService.cs, líneas 7–21.

Entre presentación y aplicación

Los Controllers reciben y almacenan Services concretos. La presentación, por tanto, depende directamente de clases del nivel de aplicación.

- AccountController → UsuarioService.
- PotreroController → PotreroService.

- ResController → ResService y PotreroService.
- UsuarioController → UsuarioService.
- VacunaController → VacunaService, ResService y PotreroService.
- VentaController → VentaService.

En estas relaciones no aparece una interfaz de Service como tipo de la dependencia. La presentación conoce directamente las clases concretas de aplicación.

Evidencia: p_mvcHacienda/Controllers/AccountController.cs, líneas 8–18; PotreroController.cs, líneas 8–20; ResController.cs, líneas 8–23; UsuarioController.cs, líneas 6–16; VacunaController.cs, líneas 9–22; VentaController.cs, líneas 7–15.

Entre presentación e infraestructura

Dos Controllers mantienen además una referencia directa a PersistenciaService: PotreroController y ResController. En ambos casos la dependencia está declarada hacia la clase concreta.

La frontera presentación–persistencia no está mediada por una abstracción propia del sistema en estas dos relaciones.

Evidencia: p_mvcHacienda/Controllers/PotreroController.cs, líneas 11–20; p_mvcHacienda/Controllers/ResController.cs, líneas 11–23.

Entre aplicación y dominio

PotreroService, ResService, VacunaService y VentaService dependen de Hacienda y, según el caso de uso, de Potrero, Res, Vacuna, Venta y sus especializaciones. Estas relaciones están declaradas mediante clases concretas del dominio.

Hacienda implementa IVacunacion, IVentaRes e ICreacionVacuna; sin embargo, los Services que usan Hacienda declaran Hacienda como tipo de campo y constructor. Por eso, la existencia de esas interfaces no cambia la dirección efectiva de las dependencias de los Services en el estado actual.

Las interfaces IVacunacion, IVentaRes e ICreacionVacuna formalizan capacidades de Hacienda, pero no son el tipo mediante el cual los Services consumen actualmente esas capacidades.

Evidencia: p_mvcHacienda/Servicios/PotreroService.cs, líneas 6–16; ResService.cs, líneas 5–15; VacunaService.cs, líneas 6–15; VentaService.cs, líneas 5–15. Bib_Hacienda/Clases/Hacienda.cs, línea 16 y operaciones asociadas a sus contratos.

Entre infraestructura y dominio

PersistenciaService usa entidades y especializaciones del dominio para guardar, cargar y reconstruir el estado: Potrero, Res, Ternero, Cebon, Novillo, Vacuna, Bacteriana, Viva, Venta y Usuario.

Aquí la dirección sí va desde un detalle técnico hacia tipos del dominio. No obstante, la dependencia está expresada hacia clases concretas del dominio, por lo que esta dirección por sí sola no se clasifica como una inversión mediada por abstracción.

Evidencia: p_mvchacienda/Servicios/PersistenciaService.cs, líneas 94–121, 132–206, 217–289, 303–378, 389–442, 454–517 y 529–589.

Dentro del mecanismo de validación

Validacion implementa IValidarInformacion y los validadores concretos heredan de Validacion. No obstante, PersistenciaService mantiene referencias concretas a ValidadorVacuna, ValidadorPotrero, ValidadorRes y ValidadorVenta para crear sus proxies.

IValidarInformacion existe como contrato del dominio de validación, pero PersistenciaService no declara sus dependencias hacia ese contrato; sus referencias son a los validadores concretos. Por ello, la presencia de IValidarInformacion describe una abstracción interna del mecanismo de validación, pero no una inversión de la frontera PersistenciaService–validadores.

Evidencia: Bib_Hacienda/Clases/Validaciones/Validacion.cs, líneas 10–17; p_mvchacienda/Servicios/PersistenciaService.cs, líneas 16–23 y 37–55.

Dependencias hacia abstracciones que sí aparecen hoy

Abstracciones técnicas del framework

- InterceptorAutenticacion e InterceptorValidarInformacion implementan IInterceptor y reciben IHttpContextAccessor.
- PersistenciaService recibe IHttpContextAccessor e IWebHostEnvironment.
- HomeController recibe ILogger<HomeController>.

Lectura AS-IS. En estas relaciones las clases concretas dependen de interfaces proporcionadas por ASP.NET o Castle. Por lo tanto, sí existe dependencia declarada hacia abstracciones técnicas. Estas relaciones pertenecen al borde técnico del sistema y no representan por sí mismas una inversión entre reglas de negocio de alto nivel y persistencia/presentación de bajo nivel.

Evidencia: Bib_Hacienda/Aspectos/InterceptorAutenticacion.cs, líneas 12–23; InterceptorValidarInformacion.cs, líneas 11–22; p_mvchacienda/Servicios/PersistenciaService.cs, líneas 12–30; p_mvchacienda/Controllers/HomeController.cs, líneas 8–16.

Contratos propios del negocio

También existen interfaces propias que formalizan capacidades del dominio: Hacienda implementa IVacunacion, IVentaRes e ICreacionVacuna; Autenticacion implementa IAutenticacion; Validacion implementa IValidarInformacion.

Estas realizaciones muestran que las clases concretas declaran conformidad con contratos propios del negocio. No significa, por sí mismo, que otras capas consuman esas clases a través de las interfaces. Para determinar el sentido real de una dependencia se observa el tipo que declara cada consumidor.

Evidencia: Bib_Hacienda/Clases/Hacienda.cs, línea 16; Bib_Hacienda/Clases/Autenticacion.cs, línea 10; Bib_Hacienda/Clases/Validaciones/Validacion.cs, línea 10.

Dónde se invierte la relación hoy

Primero, en el sentido estricto de alto nivel frente a bajo nivel: no se observa una inversión completa de la dependencia principal entre la aplicación y la persistencia. Los Services dependen directamente de PersistenciaService; los Controllers dependen de Services concretos y, en dos casos, también de PersistenciaService; y PersistenciaService usa clases concretas del dominio. Por tanto, las principales fronteras entre niveles están expresadas mediante clases concretas.

Segundo, sí se observan dependencias hacia abstracciones en relaciones específicas del código actual: los interceptores dependen de IInterceptor e IHttpContextAccessor, PersistenciaService depende de IHttpContextAccessor e IWebHostEnvironment y HomeController depende de ILogger<HomeController>. Estas son abstracciones técnicas del framework. Asimismo, Hacienda, Autenticacion y Validacion implementan interfaces propias del negocio, aunque esas interfaces no sean el tipo usado por los principales consumidores de esas clases.

Frontera	Dirección actual	¿Apunta a abstracción?	Lectura AS-IS
Aplicación ↔ persistencia	Services → PersistenciaService	No	La dependencia está declarada hacia PersistenciaService concreto.
Presentación ↔ aplicación	Controllers → Services	No	Los Controllers reciben Services concretos.
Presentación ↔ persistencia	PotreroController/ResController → PersistenciaService	No	La dependencia está declarada hacia PersistenciaService concreto.

Frontera	Dirección actual	¿Apunta a abstracción?	Lectura AS-IS
Aplicación ↔ dominio	Services → Hacienda y entidades	No en los principales consumidores	Los Services usan clases concretas del dominio.
Infraestructura ↔ dominio	PersistenciaService → entidades del dominio	No	El detalle técnico usa clases concretas del dominio.
Aspectos ↔ framework	Interceptors → IInterceptor / IHttpContextAccessor	Sí	Dependencias técnicas declaradas contra interfaces.
Persistencia ↔ framework	PersistenciaService → IHttpContextAccessor / IWebHostEnvironment	Sí	Dependencias técnicas declaradas contra interfaces.
Presentación ↔ logging	HomeController → ILogger<HomeController>	Sí	Dependencia de logging declarada contra interfaz.
Clases de dominio ↔ contratos propios	Hacienda/Autenticacion/Validacion → interfaces propias	Sí, por realización	Las clases implementan contratos propios; los consumidores principales no necesariamente usan esas interfaces.

El sistema actual combina dependencias hacia clases concretas y dependencias hacia abstracciones. La dirección principal entre aplicación y persistencia está expresada mediante PersistenciaService concreto, mientras que la presentación depende de Services concretos y, en algunos casos, directamente de PersistenciaService y Hacienda. PersistenciaService, a su vez, depende de entidades concretas del dominio para persistir y reconstruir el estado.

Las dependencias claramente declaradas hacia abstracciones se concentran en contratos técnicos del framework (IInterceptor, IHttpContextAccessor, IWebHostEnvironment e ILogger) y en las realizaciones de interfaces propias del negocio (IVacunacion, IVentaRes, ICreacionVacuna, IAutenticacion e IValidarInformacion). Por ello, al responder dónde se invierte la relación hoy, debe distinguirse entre

«dependencia hacia una abstracción» y «inversión de la frontera entre alto y bajo nivel»: el código presenta la primera en varios puntos concretos, pero no muestra una inversión completa de la relación principal aplicación–persistencia.